Agenda

- Prototype
- Class
- Array

Prototype Inheritance

- In JavaScript, prototype inheritance is a mechanism that allows objects to inherit properties and methods from other objects.
- It is the core of JavaScript's object-oriented nature, even with the introduction of the modern class syntax in ES6, which is "syntactical sugar" over the same underlying prototype system.
- Key concepts of prototype inheritance

1. Prototypes: Every JavaScript object has an internal, hidden property called [[Prototype]], which is a link to another object. That linked object is the first object's prototype.
2. Prototype chain: When you try to access a property or method on an object, JavaScript first looks for it on the object itself. If it isn't found, it searches the object's prototype. This continues up the "prototype chain" until the property is found or the end of the chain is reached. The end of the chain is Object.prototype, which has a prototype of null.
3. Inheritance is delegation: This system is sometimes described as delegation rather than traditional inheritance. A child object does not get its own copy of the parent's properties and methods. Instead, it delegates calls to the properties and methods stored on the parent object (its prototype).

Working with constructor functions

- Before the class keyword was introduced, prototype inheritance was commonly achieved using constructor functions.

```
// A "parent" constructor function
function Person(name) {
  this.name = name;
}

// Add a method to the prototype of Person
Person.prototype.display = function() {
  console.log(`Name - ${this.name}`);
};

// A "child" constructor function
function Employee(id,name, salary) {
  Person.call(this, name); // Call the parent constructor to set 'name'
  this.id = id;
  this.salary = salary;
}

// Set Employee's prototype to inherit from Person's prototype
// This creates the prototype chain: e1 -> Employee.prototype -> Person.prototype
-> Object.prototype
```

SUNBEAM INFOTECH

```
Object.setPrototypeOf(Employee.prototype, Person.prototype);

// Create a new instance
const e1 = new Employee(1, "Anil", 10000);

// The Employee object does not have its own `display` method, but it can call the
one from its prototype
e1.display(); // Output: Name - Anil
```

Key Features of Prototype

1. Underlying mechanism: Even when using the modern class syntax, JavaScript is fundamentally a prototype-based language.
2. Code reuse: Prototypes are a powerful way to share methods and properties among objects without needing to create separate copies for each instance.
3. Dynamic and flexible: The prototype chain can be modified at runtime, allowing for dynamic behavior. However, this feature should be used with caution, as it can lead to unexpected side effects.

class

- Introduced in ECMAScript 2015 (ES6), the class keyword is a syntactic sugar over JavaScript's existing prototype-based inheritance.
- While classes do not change the fundamental nature of JavaScript objects, they provide a much cleaner and more familiar syntax for creating object blueprints, inheriting from other classes, and using object-oriented programming (OOP) principles.
- Key features of JavaScript classes

1. constructor() method: A special method called automatically when a new object is created using the new keyword. It is used to initialize the object's properties.
2. new keyword: Creates a new instance of a class, triggering the constructor() method. Attempting to call a class without new will throw an error.
3. Methods: Functions defined inside the class body, which provide behavior for objects created from the class.
4. Inheritance: Classes support inheritance using the extends keyword. This allows a new class (the child) to inherit the properties and methods of an existing class (the parent).

- The super() method is used inside a child class's constructor to call the parent's constructor.

```
class Person {
constructor(name, age, addr) {
    this.name = name;
    this.age = age;
    this.addr = addr;
}
printInfo() {
    console.log("person name = ", this.name);
    console.log("person age = ", this.age);
    console.log("person addr = ", this.addr);
}
```

SUNBEAM INFOTECH

```
}
// create objects
let p1 = new Person("James Bond", 65, "London");
console.log("p1 type = ", typeof p1); // object type
p1.printInfo();

// JS class inheritance
class Employee extends Person {
constructor(name, age, addr, sal, dept) {
    super(name, age, addr);
    this.sal = sal;
    this.dept = dept;
}
showInfo() {
    super.printInfo()
    console.log("emp sal = ", this.sal);
    console.log("emp dept = ", this.dept);
}
}
```

- Static methods: Defined with the static keyword, these methods are called on the class itself, not on an instance of the class. They are useful for utility functions.

```
class Utils {
  static logMessage(message) {
    console.log(message);
  }
}
Utils.logMessage("Hello from a static method!");
```

- Private fields: Since ECMAScript 2022, private fields can be declared using a hash (#) prefix. This makes a property or method only accessible from inside the class itself, ensuring better encapsulation.

```
class BankAccount {
  #balance; // This is a private field

  constructor(initialBalance) {
    this.#balance = initialBalance;
  }

  deposit(amount) {
    this.#balance += amount;
  }

  getBalance() {
    return this.#balance;
  }
}
```

SUNBEAM INFOTECH

Array

- An array in JavaScript is a special variable that can hold multiple values.
- It allow you to store, access, and manipulate lists of data efficiently.
- Arrays are objects.
- Array object have properties (e.g. length) and methods (e.g. push(), pop(), etc).
- Array indices are in range 0 to n-1. Each element accessed using subscript.
- All array elements can be of same type (common use case) or of mixed types.
- Array declaration can be done in two ways:

1. Using Square Brackets (Recommended)
2. Using the Array Constructor

```
let fruits = ["Apple", "Banana", "Mango"];
let fruits = new Array("Apple", "Banana", "Mango");
```

- Square brackets [ ] are preferred because they are more concise.

Iterating Array

- Array can be iterated using
    1. for-loop

```
for (let i = 0; i < fruits.length; i++) {
    console.log("fruits[" + i + "] = " + fruits[i]);
}
```

    2. for-of loop values

```
for (let f of fruits) {
    console.log("element: ", f);
}
```

    3. for-in loop indexes

```
for (let i in fruits) {
    console.log("fruits[" + i + "] = " + fruits[i]);
}
```

Array of Objects

- Array of objects can be created using JSON syntax.

SUNBEAM INFOTECH

```
// Array of Objects -- using JSON syntax
 // create array
 let mobiles = [
  {
    model: "iPhone 14 Pro",
    company: "Apple",
    price: 89000.0,
  },
  {
    model: "Google Pixel 9",
    company: "Google",
    price: 92000.0,
  },
  {
    model: "Galaxy S25",
    company: "Samsung",
    price: 120000.0,
  },
];

for (let i = 0; i < mobiles.length; i++) {
    console.log(mobiles[i].model + " | " + mobiles[i].company + " | " +
mobiles[i].price);
}
```

Array Operations

1. push() - to push elements at the end.
2. splice(index, 0, newelems) - to insert elements at given index.
3. pop() - to delete the last element.
4. splice(index, delcount) - to delete num of elements from given index.
5. at() - to access element at given index like []. at() can be used with -ve index to access elements from last e.g. at(-1) is last elem, at(-2) is second last elem, ...
6. delete arr[i] - to delete element at given index - that position left "undefined".
7. slice(start,end) - to get a part of an array. the start index is inclusive, end index is exclusive.

```
//push
let names = ["Nilesh", "Rajiv", "Rohan"];
console.log("names : " + names);
names.push("Nitin");
names.push("Prashant");
console.log("names (after pushing 2 items) : " + names);

//splice - insert ops
names.splice(1, 0, "Sarang", "Vishal");
// from 1st index, delete 0 elements, and insert "Sarang", "Vishal"
console.log("names (after splicing/insert 2 items at index 1) : " + names);

//pop
```

SUNBEAM INFOTECH

```
let n1 = names.pop();
console.log("popped item : " + n1);
let n2 = names.pop();
console.log("popped item : " + n2);
console.log("names (after popping 2 items) : " + names);

//splice - delete ops
let n3 = names.splice(2, 1);
// if delete count (arg2) missing all elements after that pos are deleted
console.log("spliced/deleted item (at 2nd pos) : " + n3);
console.log("names (after splice/delete 2nd pos) : " + names);

//at
console.log("names[3] = ", names[3]);
console.log("names.at(3) = ", names.at(3));
console.log("names[-1] = ", names[-1]); // undefined
console.log("names.at(-1) = ", names.at(-1)); // access last element
// forward traversal
for (let i = 0; i < names.length; i++) {
    console.log("fwd : " + names[i]);
}
// reverse traversal
for (let i = 0; i < names.length; i++) {
    console.log(-i)
    console.log("rev : " + names.at(-i - 1));

//delete
delete names[1]; // delete element at position 1
// names[1] will be now "undefined"
// -- splice() is better than delete operator.
console.log("names (after deleting 1st pos) : " + names);

//slice
let names = ["Nilesh", "Rajiv", "Rohan", "Nitin", "Prashant"];
let part = names.slice(2, 4);
console.log("names: " + names);
console.log("names(1,4): " + part); // "Rajiv", "Rohan", "Nitin"
let part2 = names.slice(2);
console.log("names(2): " + part2); // "Nilesh", "Rajiv"
```

SUNBEAM INFOTECH